楕円曲線暗号 実装を通して学ぼう

wuhu1sland (49th)

暗号の分類

- 共通鍵暗号
  - AES
    - 使用例: Wi-Fi, SSH (暗号化)
  - ChaCha20
    - 使用例: Wireguard (暗号化)
- 公開鍵暗号
  - RSA
    - 使用例: HTTPS, SSH, VPN (署名、鍵交換など)
  - ECC(楕円曲線暗号)
    - 使用例: HTTPS, SSH, VPN (署名、鍵交換など)

電子政府における調達のために参照すべき暗号のリスト(CRYPTOREC暗号リスト)

https://www.cryptrec.go.jp/list.html

署名とは

https://ja.wikipedia.org/wiki/デジタル署名

わかりやすい図があった

鍵交換とは

=鍵共有

https://ja.wikipedia.org/wiki/ディフィー・ヘルマン鍵共有

「事前の秘密の共有無しに、盗聴の可能性のある通信路を使って、暗号鍵の共有を可能にする」こと

RSAとECCの最大の違い

安全性の根拠

RSA: 素因数分解の困難性

字面のままの意味です。RSAのアルゴリズムについてはあまり詳しくないので気になったら各自調べてください

ECC: 離散対数問題の困難性

素数 p と、p 未満の自然数 g, x を用いて

$$g^x \equiv m \mod p$$

という式で m を求めた時、x は g を底とする m の**離散対数**と呼びます

g, p, m が分かってもこの式を満たす x = 離散対数 を探すことが難しい(らしい)

ECC(楕円曲線暗号)とは

主に公開鍵暗号方式で用いられる、楕円曲線上の離散対数問題の困難性を利用した暗号の種類

代表例

- ECDH (Elliptic curve Diffie–Hellman key exchange)
- ECDSA (Elliptic Curve Digital Signature Algorithm)
- Schnorr署名
- EdDSA (Edwards-curve Digital Signature Algorithm)

これらの詳細については、あとで一つ一つ紹介します

代表的な用途

楕円曲線暗号をはじめとする公開鍵暗号方式は、主に署名や鍵交換などの用途で使用されると紹介しました。

代表例

- ECDH (Elliptic curve Diffie–Hellman **key exchange**)
- ECDSA (Elliptic Curve Digital **Signature Algorithm**)
- Schnorr**署名**
- EdDSA (Edwards-curve Digital **Signature Algorithm**)

ここからはこれらの共通のベースとなる楕円曲線上での群を作っていきます

楕円曲線

平面上での楕円曲線とは、

(Weierstrass曲線)

$$y^2 = x^3 + ax + b$$

(Twisted Edwards曲線)

$$ax^2 + y^2 = 1 + dx^2y^2$$

といった形で表される曲線です。パラメータの値はアルゴリズムごとに異なります。
また、アルゴリズムによって式の形式は異なったりもします。

(グラフを提示)

代表的な曲線例

- secp256k1
  - ECDSAなどで使用する曲線
  - Bitcoinで使用されている。Weierstrass形式で $a = 0, b = 7$
  - $y^2 = x^3 + 7$
- Ed25519
  - EdDSAなどで使用する曲線
  - ssh-keygenとかで見るやつ。Twisted Edwards形式で $a = -1, d = -\frac{121665}{121666}$
  - $-x^2 + y^2 = 1 - \frac{121665}{121666}x^2y^2$

こういった曲線の係数に加えて、実際は有限体上での演算となるため、その法となるpや、演算の起点となるベースポイントG、Gの位数nなどの情報もパラメータになります。

楕円曲線と演算の幾何的イメージ

多分図示や映像を見るのが手っ取り早い

https://www.youtube.com/watch?v=F3zzNa42-tQ&t=515s

このような加法を定義すると、無限遠点を単位元として曲線に含めることで、楕円曲線に群構造が入ります

有限体上の楕円曲線

さらに、この曲線を有限体の座標上で定義します。

素数 p を法とする整数の剰余集合 $\mathbb{Z}/p\mathbb{Z} = \{0, 1, \ldots, p-1\} = F_p$ は四則演算が閉じてる(0以外の任意の元に対して乗法の逆元が存在してる)ので、これは有限体になります

$F_p \times F_p$ の平面上で、実平面上と同じように加法を定義すると、曲線は群になります。

しかし、もはや曲線には見えず、ランダムに見える点の集合となります。

https://hypertree.seesaa.net/article/2021-07-11.html

この集合上での離散対数問題の困難性が、楕円曲線暗号の安全性の根拠となるものです。

-> 順に計算を繰り返せば $g^x \equiv m \mod p$ は計算できるが、m と g が分かっても x が復元できない

-> 曲線上の点 G に対して、$m = xG \mod n$ は計算できるが、m と G が分かっても x が復元できない (n : 素数)

楕円曲線を用いた鍵交換アルゴリズム

ECDH

Elliptic curve Diffie–Hellman key exchange / 楕円曲線Diffie–Hellman鍵交換

手順

0. Alice, Bob間で曲線やベースポイント G、その位数 n を共有しておく
1. Alice, Bobが互いにランダムな秘密の整数 $d_A, d_B \in [1, n-1]$ を定めて、$Q_A = d_A G, Q_B = d_B G$ の点の座標だけ教え合う。
2. Aliceは $d_A Q_B$、Bobは $d_B Q_A$ を計算すると、これらは同じ値 $K = d_A d_B G$ になる

Point

- ここで、d_A をAliceの秘密鍵、 Q_A をAliceの公開鍵といい、両者は**自分の秘密鍵を明かすことなく、同じ値Kを共有した**
- さらに、この二人の通信が傍受されていても、d_A と d_B のどちらも知らない第三者は K を作れない

例えば、このK(楕円曲線上の点)のx座標を共通鍵の素とすれば、信頼できない通信路上でも安全に二者間の共通鍵を作成できるようになりました。

楕円曲線を用いた署名アルゴリズム 1

ECDSA

Elliptic Curve Digital Signature Algorithm / 楕円曲線デジタル署名アルゴリズム

SSL証明書やBitcoinの署名など、幅広く使われている楕円曲線暗号署名のスタンダードです。1999年にNISTにより標準化されました。

手順

Alice (署名者)

0. Alice, Bob間でパラメータを決めておく
1. Aliceが秘密鍵 $d_A \in [1, n-1]$ を作って、公開鍵 $Q_A = d_A G$ を計算
2. メッセージ m のハッシュ値を e とする ($e = H(m)$)
3. 位数nのビット長分だけ e の上位を取り出して、この値を z とする
4. 乱数 $k \in [1, n-1]$ を作って、$(x, y) = kG$ を計算
5. $r = x \mod n$ と $s = k^{-1}(z + rd_A) \mod n$ を計算
6. (r, s) を署名として、メッセージmと一緒にBob(検証者)に送信

ここで、kは**nonce**と言い、内部的な秘密鍵になります。また、kは$\mathbb{Z}/n\mathbb{Z}$の元になっているので、$[1, n-1]$内にk^{-1}が存在します。

Bob (検証者)

7. (r, s) を受け取る
8. Aliceと同様の手順でメッセージ m のハッシュ値 e とその上位部分 z を計算
9. $u_1 = zs^{-1} \mod n$、$u_2 = rs^{-1} \mod n$ を計算
10. $(x', y') = u_1 G + u_2 Q_A$ を計算
11. $x' \equiv r (\equiv x) \mod n$ なら、正当な署名

公開情報

- パラメータ: 素数 p, ベースポイント G, G の位数 n など
- Aliceの公開鍵 Q_A
- 送信されたメッセージ m と、署名 (r, s)

手計算で実際に変数を展開していくと検証される様子がわかって気持ち良いです。

余談 PS3事件

SONYはプレステ3のソフトウェア署名にECDSAを採用していましたが、全ての署名で同じnonce kを使い回してしまいました。

その結果、George Hotzを始めとするハッカーコミュニティが、その脆弱性から署名用秘密鍵d_Aを逆算することに成功するという事件が2010年に発生しました😇

なぜ秘密鍵が漏れるのか

ECDSAでは署名(r, s)の$r = x \mod n$（ここで$(x, y) = kG$）はkだけに依存します。

同じkを2回使うと、当然rは同じ値になります。

2つの異なるメッセージm_1, m_2（ハッシュ値のマスクをz_1, z_2）に対して、同じkで署名を作ると

$$s_1 = k^{-1}(z_1 + rd_A) \mod n$$

$$s_2 = k^{-1}(z_2 + rd_A) \mod n$$

この連立方程式から、未知の変数はkとd_Aの2つですが、方程式が2つあるため、解くことができます：

$$k \equiv \frac{z_1 - z_2}{s_1 - s_2} \mod n$$

kが分かれば、sの式をd_Aについて解いて

$$d_A = \frac{sk - z}{r} \mod n$$

署名用の d_A 割れたことで、所謂jailbreakが可能になりました。

この固定化されたnonceの特定に成功したのが、marcan, bushing, svenの三人のチームで、そこからソフトウェア署名用の鍵を割ったのがGeorge Hotz (と認識しているけど間違ってるかも)

ちなみに、marcan, bushingはWiiの非公式ゲームランチャーとして有名な「Home brewchannel」の開発メンバーで、svenはApple SiliconのMacで動作するLinuxカーネル移植プロジェクト「Asahi Linux」の開発メンバーです。

George Hotzは17歳でiPhoneのjailbreakに個人として初めて成功したハッカーで、現在は自動運転開発のComma.aiとTinygradsというDL用フレームワークを開発するTiny Corporationを立ち上げています。

余談2 Snowden事件

映画とかにもなってるアレです。

NIST-p256などの楕円曲線の乱数生成器としてNSAが推奨していたDual_EC_DRBGというアルゴリズムに、バックドアが仕込まれていたのではないかという疑惑が生まれ、Dual_EC_DRBGの使用は非推奨になりました。

この乱数生成器も、二つのベースポイントの離散対数問題が予測不可能性の根拠となっているのですが、そのベースポイントの決定根拠が不明瞭で、NSAが事前に解となるスカラー値を把握しているポイントをあえて設定したのではないかと疑われています。

乱数生成の生成パターンが予想可能になると、先ほどのプレステの脆弱性と同様、nonceなどの値が割れてしまい、結果的に秘密鍵の復元などに使われかねません。

正直、この脆弱性が意図的なのかどうかは私には判断しかねますが、自分が暗号系の知識を習ったバイト先の上司の人は割としっかりバックドアだと主張していたので、黒寄りだと認識している人が多いよう?です

脆弱性について詳しくはこれを参考にしてください

https://techmedia-think.hatenablog.com/entry/2020/10/14/235117

楕円曲線を用いた署名アルゴリズム２

Schnorr署名

1989年にClaus P. Schnorrによって発明された署名アルゴリズムで、2008年まで特許の都合であまり利用されていませんでした。

ECDSAより検証の効率が良いことや、集約署名が可能であるなどの性質から、特許が切れてからは、ECDSAに代わる署名方式として注目されていました。

現在はその上位互換みたいなEdDSA(後述)の方が注目されてるっぽいです。

手順 (BIP-340)

Alice (署名者)

1. Aliceが秘密鍵 d_A と 公開鍵 $Q_A = d_A G$ を用意
2. 乱数 $r \in [1, n-1]$ を用いて $R = rG$ を計算
3. メッセージ m と公開鍵 Q と R を元にハッシュ計算 $H(R||Q_A||m)$
4. $s = r + H(R||Q_A||m)d_A \mod n$ を計算して、(R, s) を署名として送信

Bob (検証者)

5. (R, s) を受け取る
6. $sG = R + H(R||Q_A||m)Q_A$ が成り立てば正当な署名

めちゃくちゃシンプルです。証明も簡単。

楕円曲線を用いた署名アルゴリズム３

EdDSA

2011年に発表された、比較的新しいアルゴリズムです。
注意すべきなのは、今までのWeierstrass形式の曲線ではなくtwisted Edwards曲線を用いているところです。

手順

Alice(署名者)

1. 秘密鍵 d_A のハッシュ $H(d_A)$ の前半を s とし、 公開鍵 $Q_A = sG$ を計算
2. 秘密鍵 d_A のハッシュ $H(d_A)$ の後半を prefix とする
3. $r = H(\mathrm{prefix}||m) \mod n$ として、$R = rG$ を計算
4. $k = H(R||Q_A||m) \mod n$ を計算
5. $S = (r + ks) \mod n$ として、(R, S) をBobに送信

Bob(検証者)

6. (R, S) を受け取る
7. $k = H(R||Q_A||m) \mod n$ を計算
8. $8(SG) = 8(R + kQ_A)$ が成立すれば、正当な署名

これもシンプルですね

schnorr署名との違いとして

1. nonce r を決定論的に計算 = 乱数生成器を用いていない
2. 8という謎の値(cofactor)が登場する

という点があります

1. nonce r を決定論的に計算 = 乱数生成器を用いていない

プレステやSnowdenの話の中で述べたように、アルゴリズム内で用いられるnonceおよび、それを生成する乱数生成器は暗号の安全性に関わります。

EdDSAでは秘密鍵のハッシュから決定論的に計算されるので、乱数生成器の安全性に依存しないアルゴリズムが実現されています

2.8という謎の値(cofactor)が登場する

実はECDSAやSchnorr署名とは異なり、EdDSAで用いられる曲線の位数は素数ではなく、素数 n を用いて $8n$ と表せる8の倍数になっています。(この n は G の位数 n と同じ値)
これは曲線としてtwisted Edwardsを採用したことによる副作用です。

twisted Edwards曲線を採用したことによって、EdDSAでは完全加法公式を使用できるというメリットがあります。

完全加法公式

$P = (x_1, y_1), Q = (x_2, y_2)$ の二点の加算 $P + Q = (x_3, y_3)$ を考えるとき、

- twisted Edwards形式の加法公式（完全加法公式）

  $$(x_3, y_3) = \left(\frac{x_1 y_2 + x_2 y_1}{1 + d x_1 x_2 y_1 y_2}, \quad \frac{y_1 y_2 - a x_1 x_2}{1 - d x_1 x_2 y_1 y_2} \right)$$

- Weierstrass形式の加算公式

  $x_3 = \lambda^2 - x_1 - x_2$

  $y_3 = \lambda(x_1 - x_3) - y_1$

  - P ≠ Q の場合
    - $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$
  - P = Q の場合(2倍算)
    - $\lambda = \frac{3x_1^2 + a}{2y_1}$

このように、twisted Edwards形式では点による場合分けが不要になるので、任意の点の加算処理に差が出ません。これはサイドチャネル攻撃への耐性を強化します。

しかし、完全加法公式を成立させるtwisted Edwards曲線では位数が $8L$ になってしまうので、これによって生じる小位数部分群の影響をキャンセルするため(厳密には、不正な署名が誤って正当だと検証されることを防ぐために)にcofactor 8をかけるという処理が必要になっています。

パフォーマンスのためにcofactorlessな検証を行っている(検証の最後に8をかけて小位数部分群をキャンセルしない)実装も多いようですが(https://eprint.iacr.org/2020/1244.pdf)(https://www.researchgate.net/publication/392448016_On_cofactored_verification_of_EdDSA_signatures)、これに起因する脆弱性がMoneroなどで発見されたりしていました。(https://techmedia-think.hatenablog.com/entry/2018/12/05/195659)

応用例の紹介 1

ssh

ざっくり

1. 鍵交換で共有秘密を作成して、そこから暗号化に使用する共通鍵を作成
2. サーバはホスト鍵で自分を証明。クライアントは~/.ssh/known_hostsと照合 or 登録
3. クライアントは鍵交換で確立されたセッションIDを秘密鍵で署名して送信
4. サーバは公開鍵で検証。対応する公開鍵があれば認証

応用例の紹介 2

Bitcoin

Bitcoinの(一般的な)支払いの流れ

- ブロックチェーン = Bitcoinネットワーク参加者が全員保持する履歴ファイル

1. UTXO(Unspent Transaction Output)をアンロック

- ブロックチェーン上で、(未使用の)自分宛送金トランザクションをUTXOという
- 自分宛の支払いには、自分の持ってる秘密鍵に対応した公開鍵のhashが載っている
- それぞれのUTXOに対して秘密鍵で署名を作成

2. トランザクションの作成と送信

- 署名と公開鍵つきUTXOをinputとして、送信相手の公開鍵のhashをoutputとして登録したトランザクションを作成
- トランザクションを他のBitcoinノードに送信

3. 検証

- マイナーが新たなトランザクションを受け取ると、公開鍵のhash計算とinputのUTXOの署名を検証
- 正当なトランザクションであればプールに保持

4. PoW

- マイナーはプールからトランザクションを集めて、トランザクション束 + nonceのhash値が条件を満たす(「先頭nビットが連続で0になる」など)までnonce入れ替えとhash計算を行い、条件を満たせばブロックとして他ノードに送信
- ちなみに、このブロックの先頭にはcoinbaseトランザクションと呼ばれる特殊なトランザクションが含まれる。これはinputを持たず、マイニング報酬 + ブロック内全トランザクションの手数料の合計を自分宛に出力するもので、ルール上1ブロックに1つだけ許可されている

5. ブロックチェーンへの追加

- 他ノードはブロックの検証をし、正当であれば各々ブロックチェーンに追加

このフローの中で使用される署名アルゴリズムはsecp256k1という曲線のECDSAです。

Taproot

2021年にBitcoinに導入されたアップグレードです。

従来のBitcoinと大きく異なるのは、

- outputに公開鍵のハッシュの代わりに、公開鍵のx座標を記録
- ECDSAの署名ではなく**Schnorr署名**を使用

という点です。これによって、Schnorr署名の線形性を活用したマルチシグなどが実現できます。また、署名のバッチ検証が可能になったことや、アンロック時に付与するのが署名だけで良くなったことなどから、パフォーマンスも向上します。

さらに、従来と同様の支払い方法(key-path spend)に加えて、マークルツリーを用いたscript-path spendという支払い方法も実装されています。これも面白いので興味があれば調べてみてください。

演習

ECDH、ECDSA、Schnorr、EdDSA等、気になったものを実装してみましょう

`/src/elixir` には、ElixirでECDSAとECDHを実装したものが入っています。

super useful参考文献

O'Reillyの『プログラミング・ビットコイン』および、そのサンプルコードリポジトリ

https://github.com/jimmysong/programmingbitcoin